

REAL-TIME RENDERING 4th Study

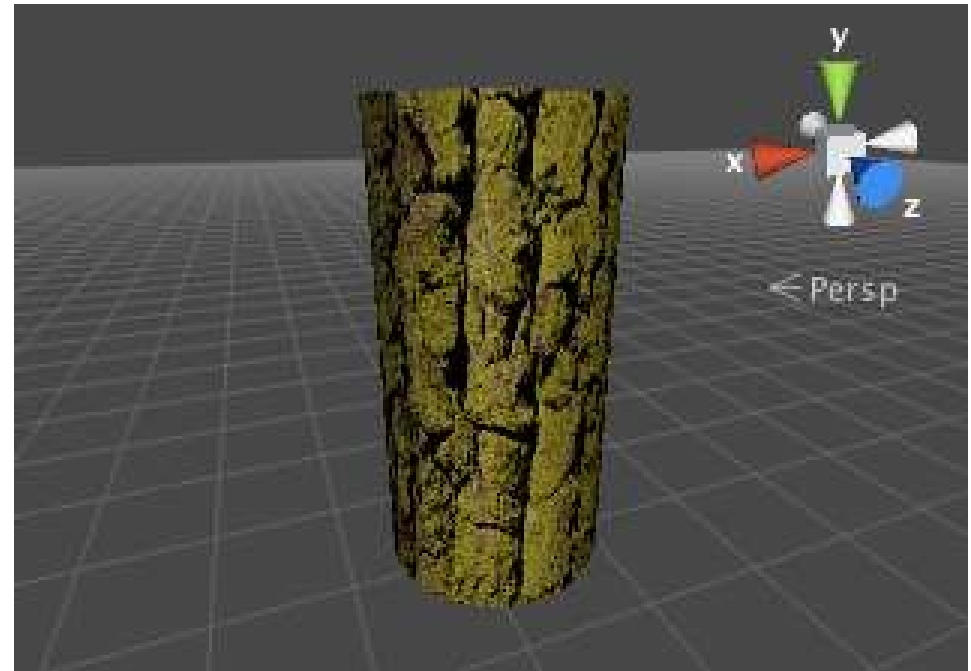
Chapter 6. Texturing

최장현

목차

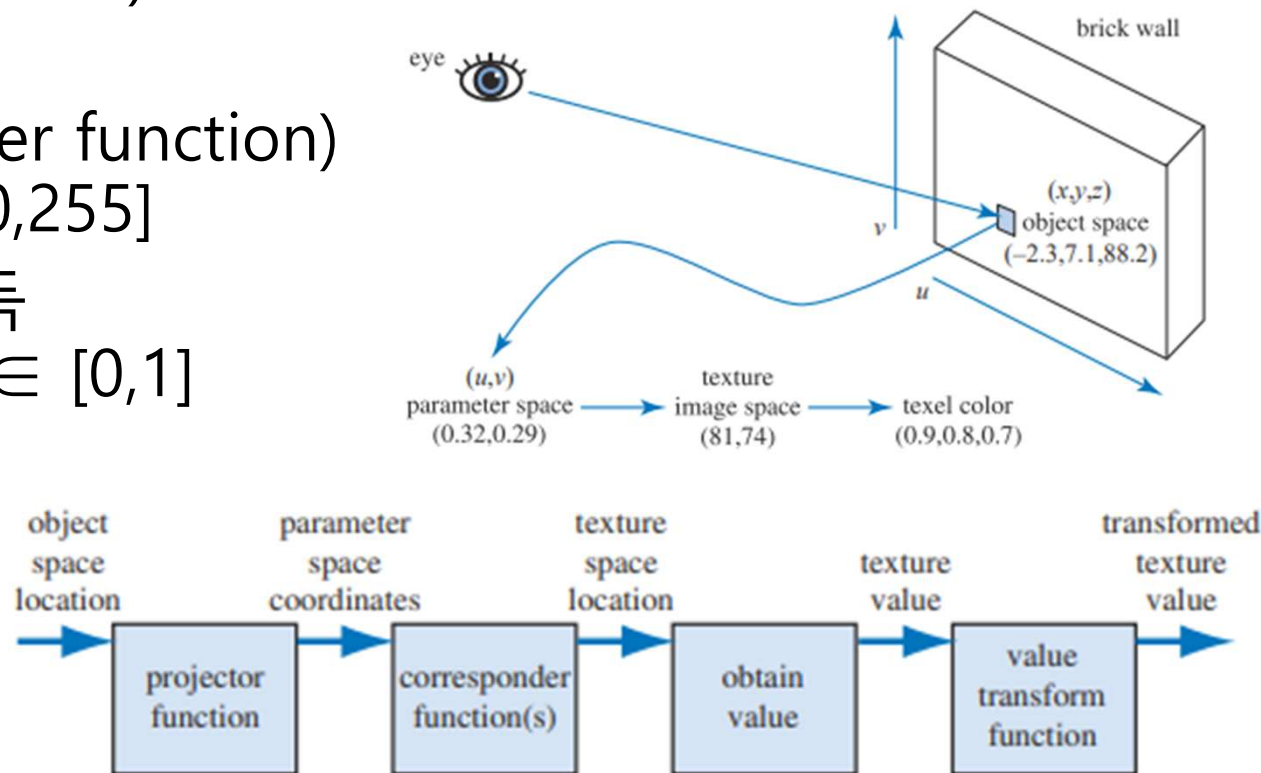
- 6.1 텍스처 처리 파이프라인
- 6.2 이미지 텍스처 처리
- 6.3 절차적 텍스처 처리(Procedural Texturing)
- 6.4 텍스처 애니메이션
- 6.5 재질 매핑(Material Mapping)
- 6.6 알파 매핑(Alpha Mapping)
- 6.7 범프 매핑(Bump Mapping)
- 6.8 시차 매핑(Parallax Mapping)
- 6.9 텍스처 조명(Textured Lights)

Texturing?



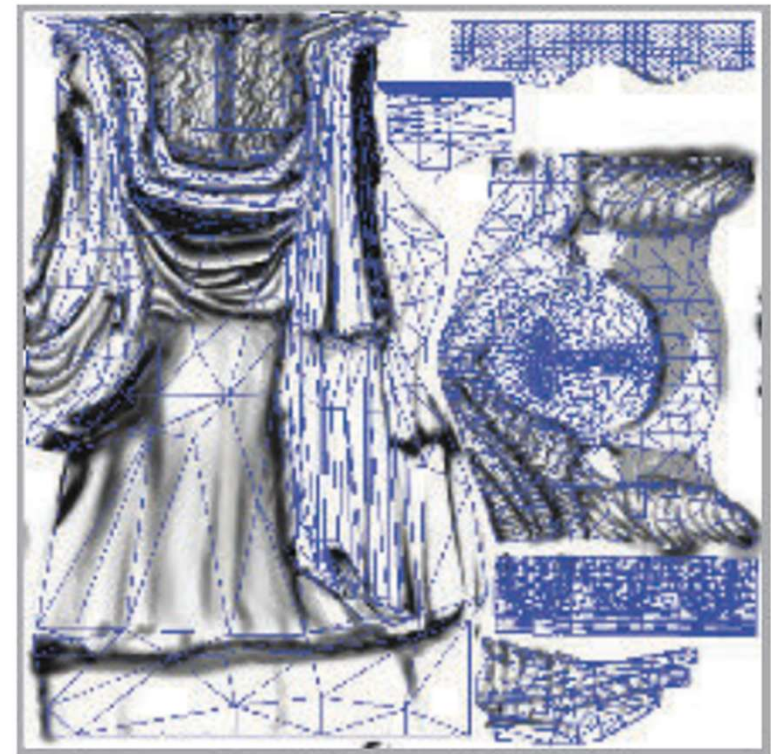
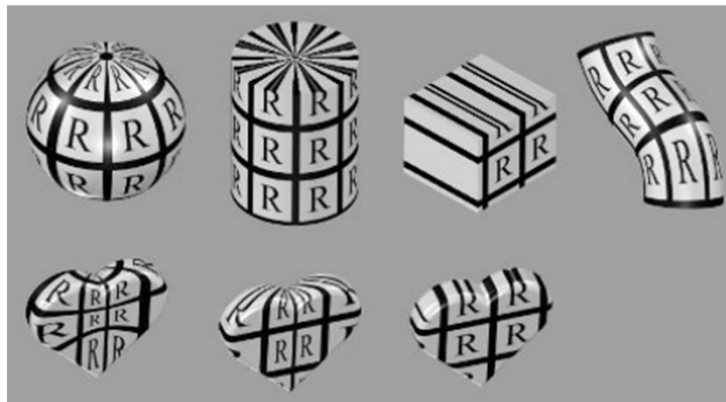
6.1 텍스처 처리 파이프라인

1. 투영 함수(projector function)
 - $x, y, z \rightarrow u, v$
2. 대응자 함수(corresponder function)
 - $u, v \in [0, 1] \rightarrow u, v \in [0, 255]$
3. (u, v) 의 텍셀 컬러 값 획득
 - $u, v \in [0, 255] \rightarrow r, g, b \in [0, 1]$
4. 컬러 값 선형 변환
 - sRGB $\rightarrow$ Linear RGB
5. 음영 계산에 사용



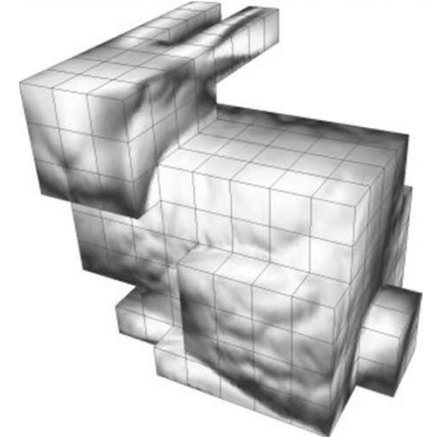
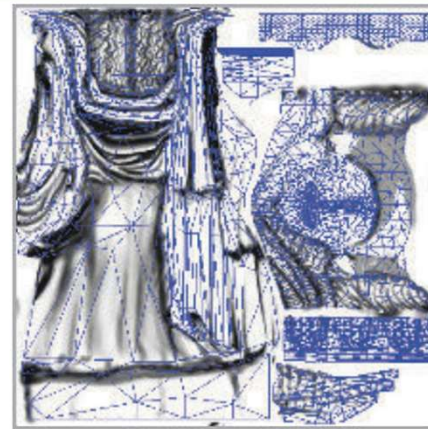
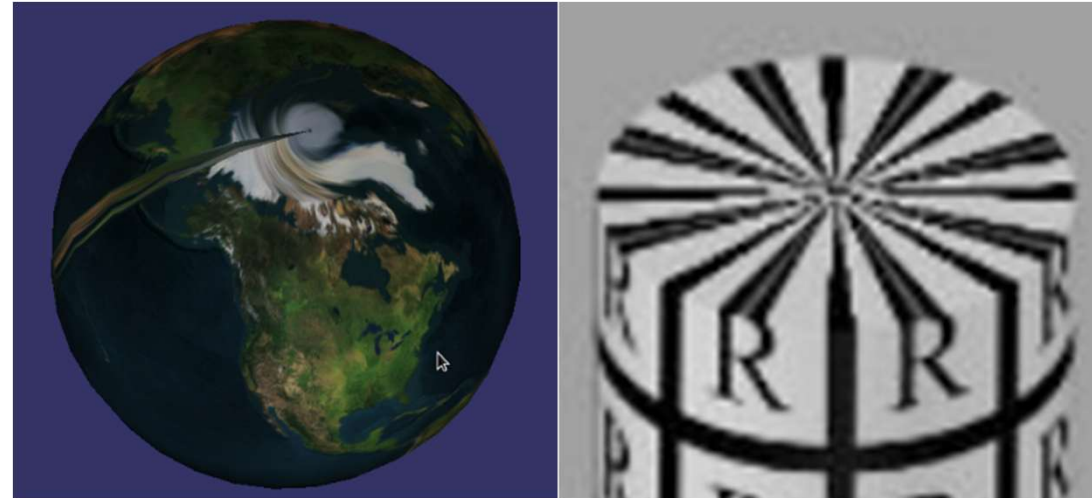
6.1 텍스처 처리 파이프라인 – 투영 함수

- 투영 함수(projector Function)
- 목표 : 텍스처 좌표 생성
- 생성 방법
 - 대표적인 투영 방식 : 구, 원통, 평면 등
 - 메시 언래핑(mesh unwrapping)
 - > 메시 파라미터화(mesh parameterization)



6.1 텍스처 처리 파이프라인 – 투영 함수

- 왜곡(Distortion)
 - 문제
 - 면 경계(seam) 불연속
 - 투영 방향 수직인 면
 - 해결
 - 메시 언래핑 및 파라미터화
 - 경계 블렌딩
 - 폴리큐브 맵(polycube map)



6.1 텍스처 처리 파이프라인 – 투영 함수

- 다양한 투영의 입력과 형태
 - (u, v, w) 시스템
 - w : 투영 방향 깊이
 - volume texture
 - (s, t, r, q) 시스템
 - q : 동차 좌표계 값(homogeneous coordinate)
 - 표면의 법선의 방향에 따라 투영
 - cube map
 - 1차원 텍스처
 - 고도 등

6.1 텍스처 처리 파이프라인 – 대응자 함수

- 대응자 함수(corresponder function)
 - 매개변수 공간 값을 텍스처 공간 값으로 변환
 - 텍스처 적용의 유연성 제공
- 주요 변환 기능
 - 행렬 변환 : 이동, 회전, 확대, 축소, 전단, 투영 등
 - 이미지 적용 방식 제어 : $u, v \in [0, 1]$ 범위 외 처리
 - 해상도 기반 위치 계산 : $u, v \in [0, 1] * \text{해상도}$
 - 값 변경 없이 해상도가 다른 이미지 텍스처 교체 가능

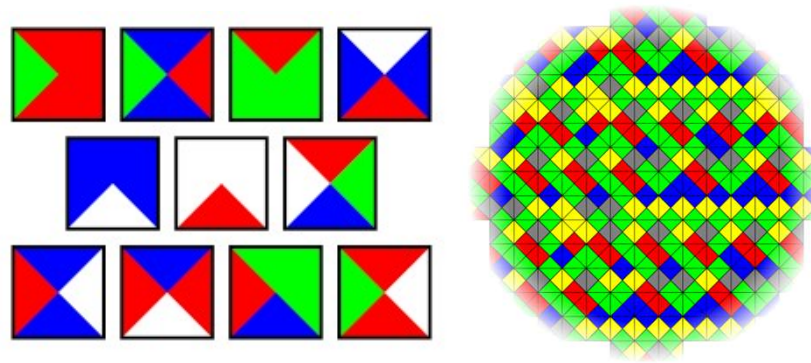
6.1 텍스처 처리 파이프라인 – 대응자 함수

- 이미지 적용 방식 제어
- wrap / repeat / tile : 반복
- mirror : 반복 뒤집기
- clamp / clamp to edge : 경계 픽셀 늘리기
- border / clamp to border : 단색 경계



6.1 텍스처 처리 파이프라인 – 대응자 함수

- 주기성 문제 : 3회 이상 반복 시 인지 가능성 높음
- 해결
 - 다른 비반복 텍스처와 결합
 - 텍스처 패턴 무작위 재조합 (Wang tiles 등)
 - 종속 텍스처 읽기(dependent texture read) : 한 텍스처에서 데이터를 읽고 그 결과를 사용하여 다음 텍스처의 주소를 지정하는 것



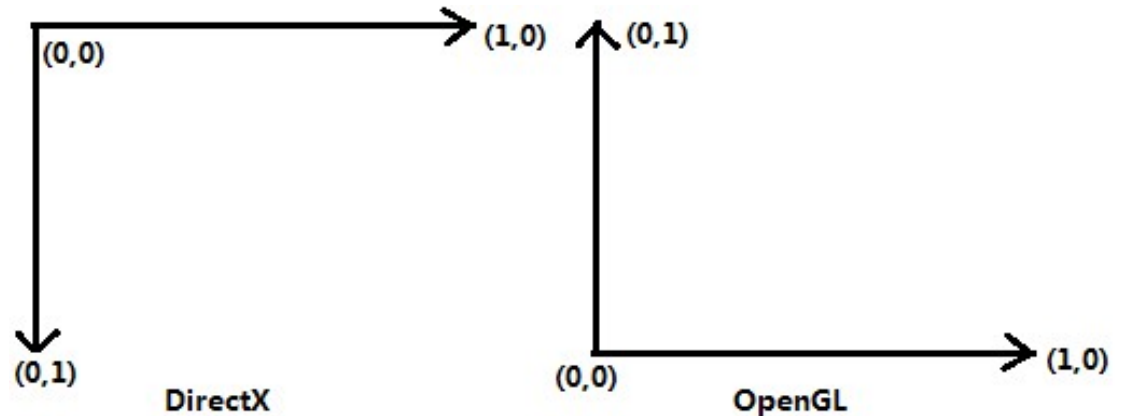
Wang tiles

6.1 텍스처 처리 파이프라인 – 텍스처 값

- 텍스처 값(Texture Values) : 대응자 함수로 생성한 텍스처 공간 좌표 -> 텍스처 값 추출
- 텍스처 유형
 - 이미지 텍스처
 - 절차적 텍스처(Procedural Texture)
- 텍스처 값 종류
 - RGB
 - Grayscale
 - RGBa
 - 법선
 - bump map

6.2 이미지 텍스처 처리

- 2D 이미지를 폴리곤 표면에 붙이는 방식
- 텍스처 좌표 -> 텍셀 컬러 값 추출
- 삼각형 내부 엘리어싱 문제
- API별 텍스처 좌표
 - DirectX
 - (0,0) : 좌상단
 - (1,1) : 우하단
 - OpenGL
 - (0,0) : 좌하단
 - (1,1) : 우상단
- 텍셀 중심 좌표 표준(현재 표준) : (0.5, 0.5)
- Flooring 방식 : $[(n).0, (n+1).0) \rightarrow n$



6.2 이미지 텍스처 처리

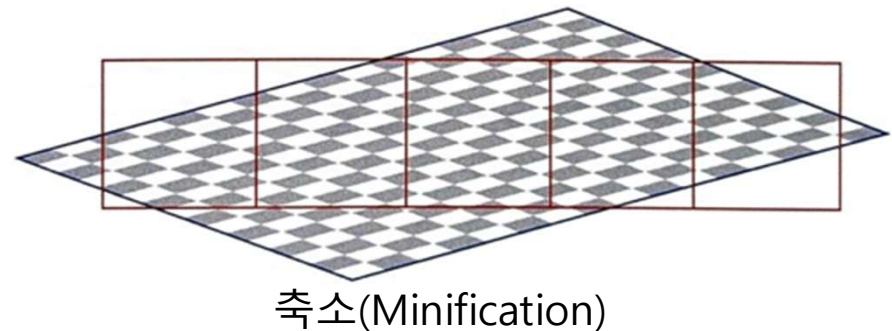
- 종속 텍스처 읽기(Dependent Texture Read)
- 정의 1) 픽셀 셰이더 내에서의 좌표 수정 (모바일 위주)
 - 버텍스 셰이더에서 전달받은 uv 좌표를 그대로 쓰지 않고, 픽셀 셰이더 내에서 계산을 통해 수정한 uv로 텍스처를 읽는 경우
 - 구형 모바일 GPU에서는 prefetch 기능을 방해해 성능이 떨어질 수 있음
- 정의 2) 첫 번째 텍스처에서 읽은 값을 이용해 두 번째 텍스처의 좌표를 결정함(데스크톱 위주)
 - 노멀 맵에서 법선 정보를 읽고, 해당 정보를 바탕으로 큐브맵의 반사 좌표 계산

6.2 이미지 텍스처 처리

- 텍스처 크기 제약
- 과거
 - POT(power-of-two) 텍스처
 - $2^m \times 2^n$
- 최신
 - NPOT(non-power-of-two) 텍스처 지원
- 일부 구형 모바일 GPU는 NPOT+mipmapping 미지원
- GPU마다 텍스처 크기 상한 다름(DirectX 12는 16384^2)

6.2 이미지 텍스처 처리

- 확대(Magnification) - ex) 아주 가까운 오브젝트
- 축소(Minification) - ex) 아주 먼 오브젝트
- 각각 맞는 샘플링과 필터링 필요
- 입출력이 선형 관계일 때 필터링 유효
 - 노멀, 거칠기는 비선형관계로 기본 필터링으로 앨리어싱 발생



6.2 이미지 텍스처 처리 - 확대

- 확대(Magnification)



최근접 이웃 필터링
(Nearest-neighbor filtering)



이중 선형 필터링
(Bilinear filtering)



바이큐빅 필터링
(Bicubic filtering)

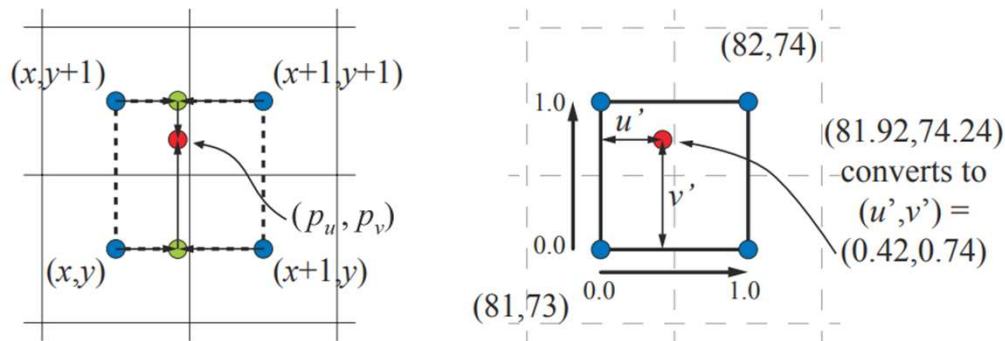
6.2 이미지 텍스처 처리 - 확대

- 최근접 이웃 필터링(Nearest-neighbor filtering)
 - 각 픽셀 중심으로부터 가장 가까운 텍셀 값을 그대로 사용
 - 간단하고 비용 낮음
 - 픽셀화(pixelation) 발생



6.2 이미지 텍스처 처리 - 확대

- 이중 선형 필터링 (Bilinear filtering)
 - 가장 가까운 4개의 텍셀을 사용해 선형 보간
 - 샘플에 가까운 텍셀이 더 큰 영향
 - 앨리어싱 감소
 - 블러 발생

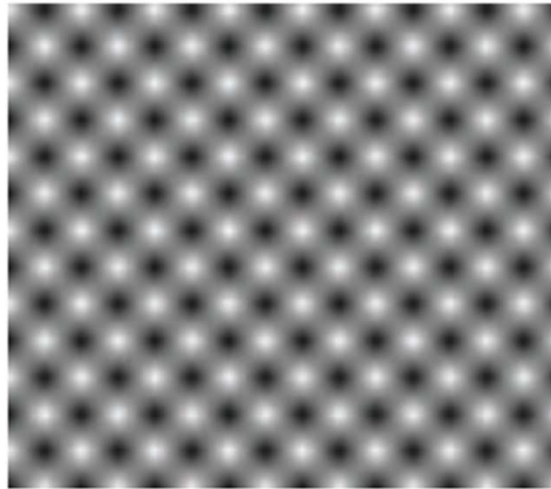


$$\begin{aligned} b(p_u, p_v) &= (1 - v')((1 - u')t(x, y) + u't(x + 1, y)) \\ &\quad + v'((1 - u')t(x, y + 1) + u't(x + 1, y + 1)) \\ &= (1 - u')(1 - v')t(x, y) + u'(1 - v')t(x + 1, y) \\ &\quad + (1 - u')v't(x, y + 1) + u'v't(x + 1, y + 1). \end{aligned}$$

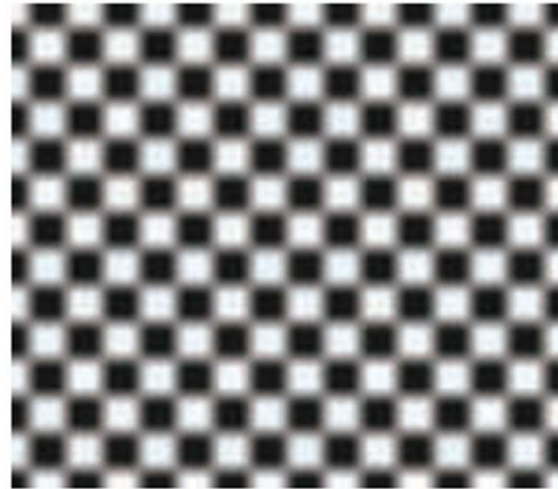


6.2 이미지 텍스처 처리 - 확대

- 블러 개선
 - 리매핑
 - 디테일 텍스처(detail textures)



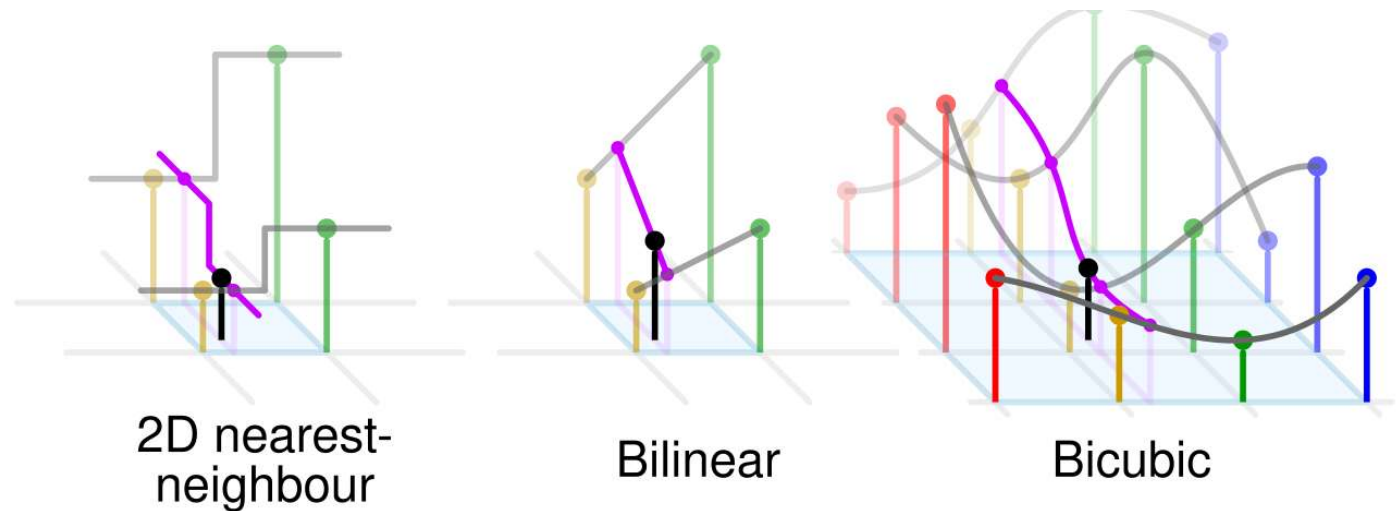
bilinear filtering



remapping

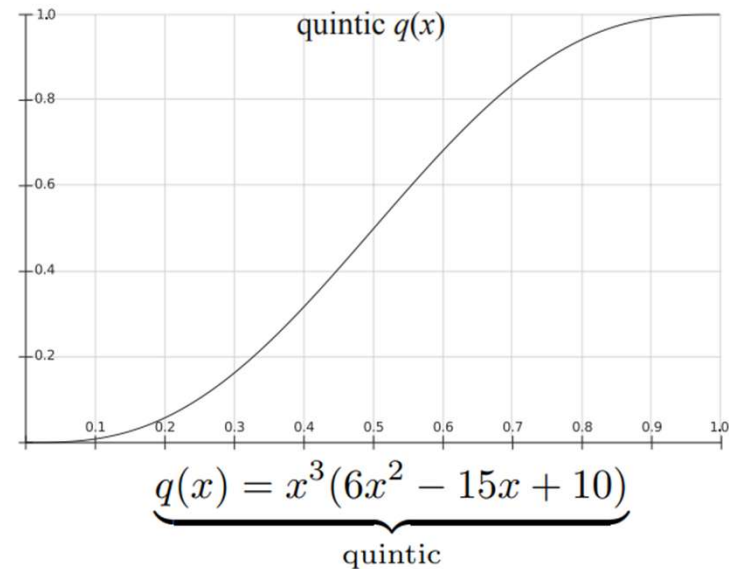
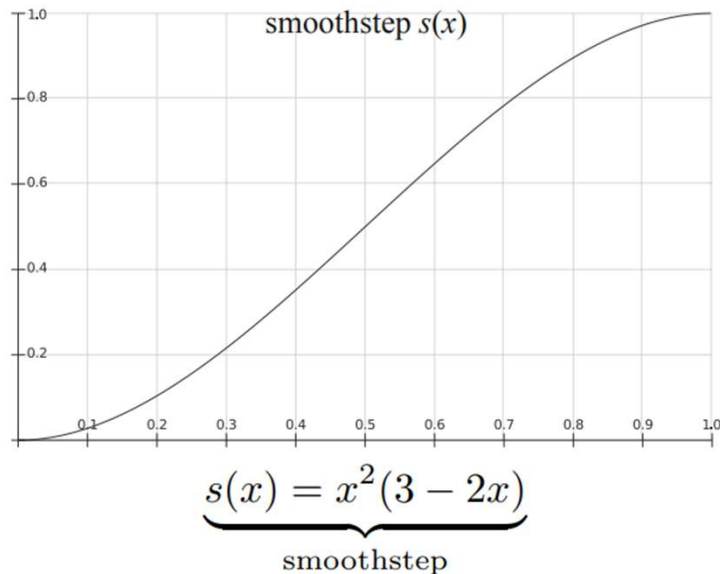
6.2 이미지 텍스처 처리 - 확대

- 바이큐빅 필터링(bicubic filtering)
 - 비용 높음
 - 고차 선형 보간
 - 하드웨어 내장 바이리니어 보간 기능 여러 번 호출 활용 가능

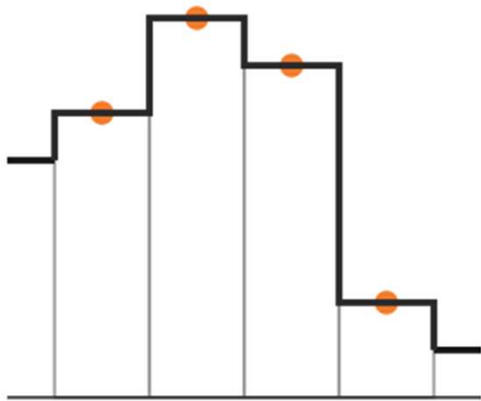


6.2 이미지 텍스처 처리 - 확대

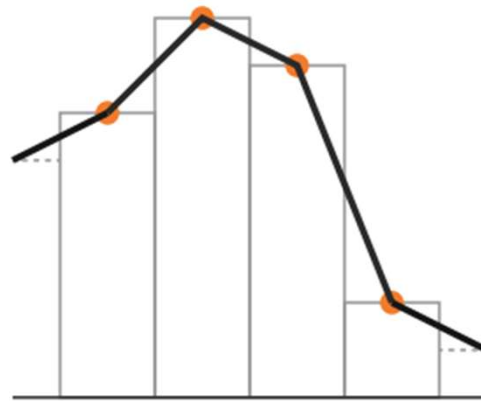
- 부드러운 곡선(smooth curve)
 - 2x2 텍셀 세트 사이 보간
 - $s'(0) = s'(1) = 0$ / $q''(0) = q''(1) = 0$
 - bilinear + uv 값에 필터링
 - ex) $u=0.1, q(u)=0.028, v=0.5, q(v) = 0.5$



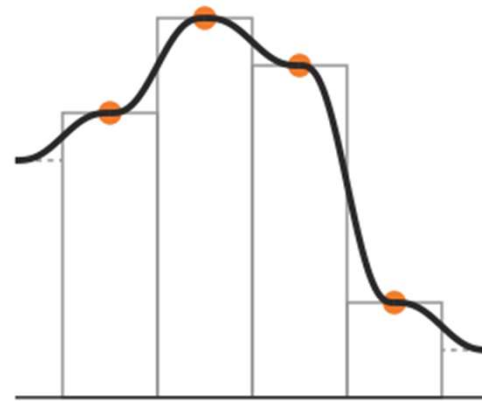
6.2 이미지 텍스처 처리 - 확대



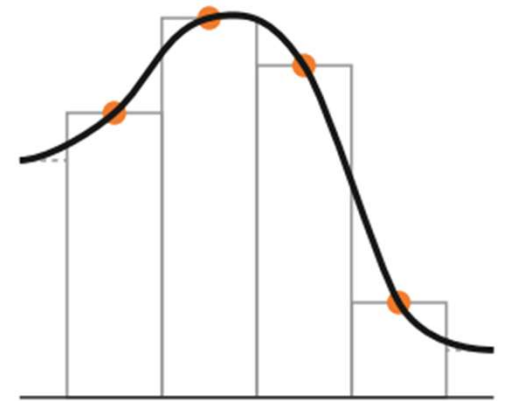
Nearest-neighbor filtering



Bilinear filtering



quintic curve

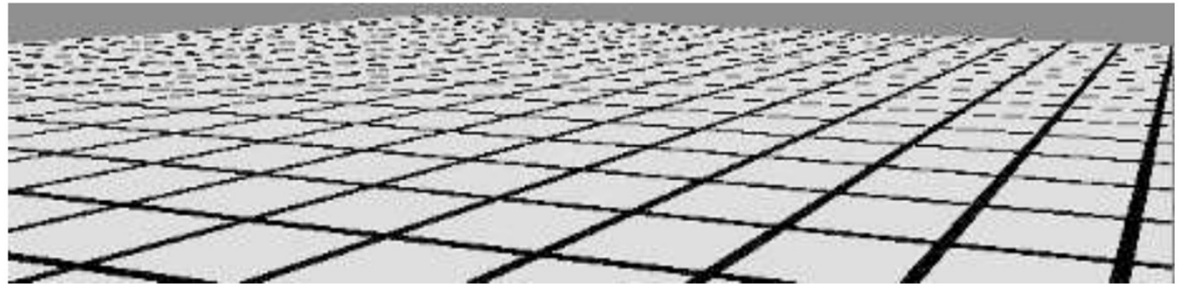


Bicubic filtering

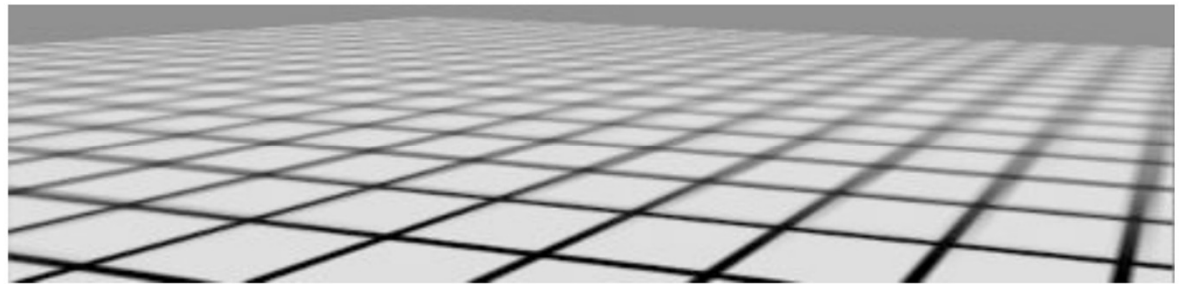
6.2 이미지 텍스처 처리 - 축소

- 축소(Minification)

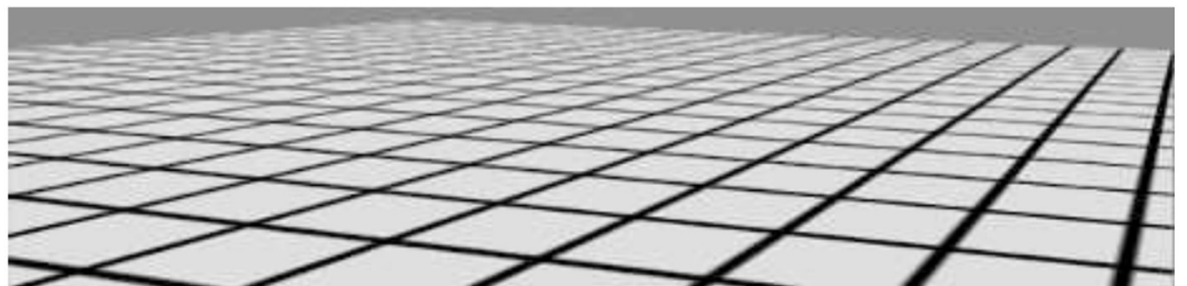
최근접 이웃 필터링
(Nearest-neighbor filtering)



미프맵핑
(Mipmapping)

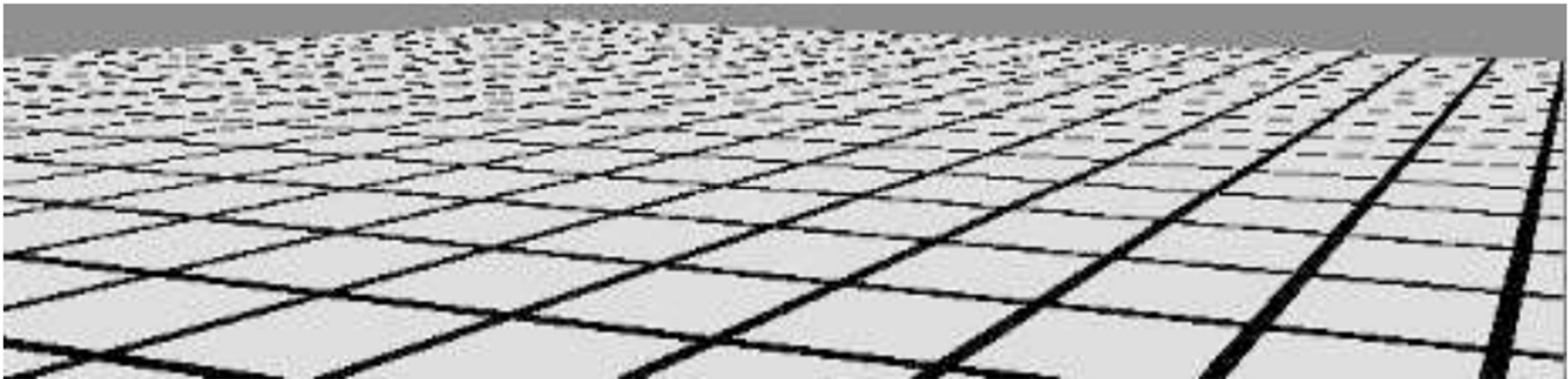


합산 영역 테이블
(Summed area table)



6.2 이미지 텍스처 처리 - 축소

- 최근접 이웃 필터링(Nearest-neighbor filtering)
 - 각 픽셀 중심으로부터 가장 가까운 텍셀 값을 그대로 사용
 - 간단하고 비용 낮음
 - 앨리어싱 발생
 - 시간적 앨리어싱(temporal aliasing) 심화
- 이중 선형 필터링(Bilinear filtering) / 샘플링 및 필터링

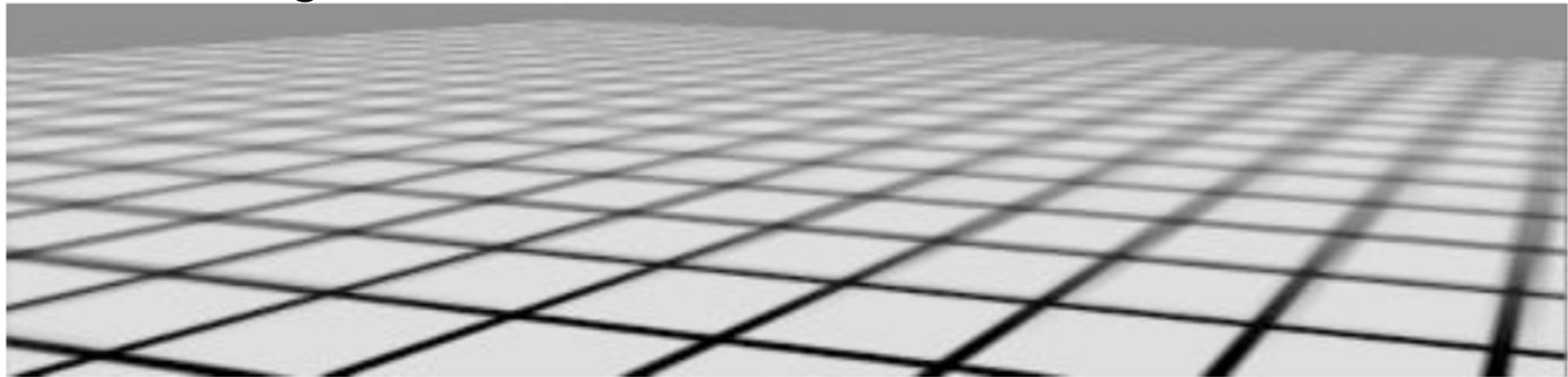
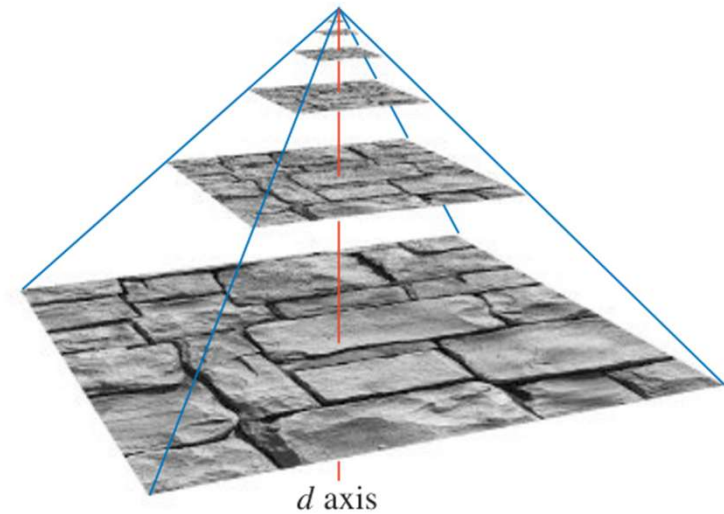


6.2 이미지 텍스처 처리 - 축소

- 이중 선형 필터링(Bilinear filtering)
 - 픽셀당 텍셀 4개 초과 -> 앨리어싱
- 샘플링 및 필터링
 - 나이퀴스트 한계(Nyquist limit) : 텍스처 신호 주파수가 샘플링 주파수의 절반보다 커지지 않도록 -> 픽셀당 1개의 텍셀
 - 픽셀 샘플링 주파수 높이기 : SSAA, MSAA .. (Chapter 5)
 - 비용 문제
 - 축소의 한계
 - 텍스처 주파수 낮추기
 - mip매핑(mipmapping)

6.2 이미지 텍스처 처리 - 축소

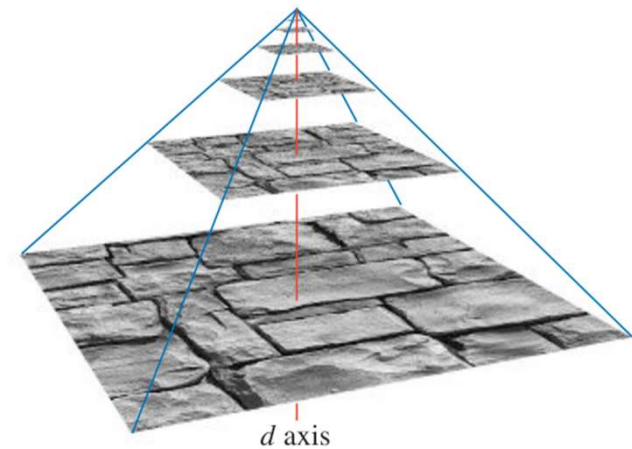
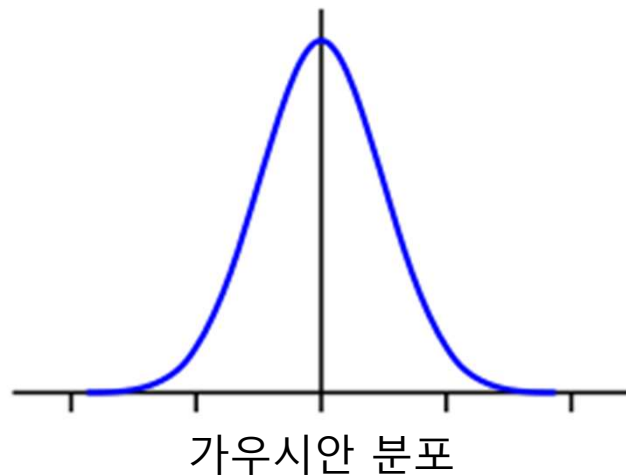
- mip매핑(Mipmapping)
 - 레벨 0(원본)부터 가로세로 절반씩 줄여 하위 이미지 생성
 - 한 변이 1텍셀이 될 때까지 반복 -> mip맵 체인
- 고품질 mip맵
 - 필터링 방법
 - 감마 보정(gamma correction)



6.2 이미지 텍스처 처리 - 축소

- mip맵 필터링

- 2x2 평균(Box filter) -> 앨리어싱 유발
- 가우시안(Gaussian), 란초스(Lanczos), 카이저(Kaiser) 등 ..
- 가우시안(Gaussian filter) : 중심에 가까운 텍셀 높은 가중치



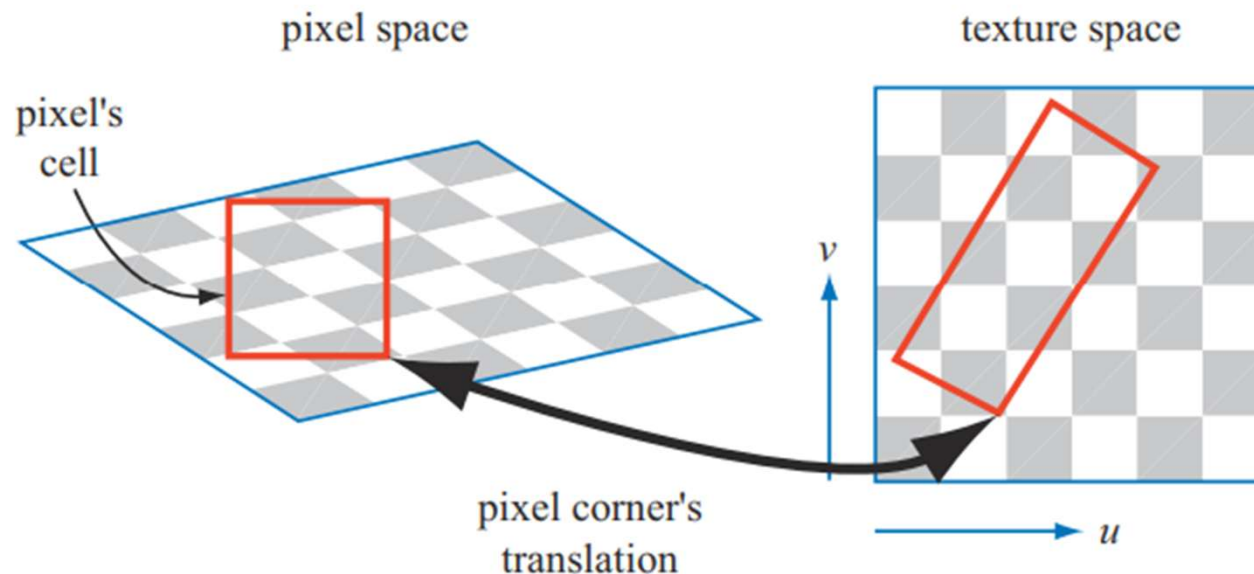
6.2 이미지 텍스처 처리 - 축소

- mip맵 감마 보정(Gamma correction)
 - 텍스처는 보통 비선형 공간(sRGB)
 - 필터링시 밝기와 대비 등 디테일 변형 가능(ex : 어두워짐)
 - sRGB -> 선형공간변환 -> 필터링 -> sRGB



6.2 이미지 텍스처 처리 - 축소

- mip맵 작동 원리 : d(or λ) 결정 (Level of Detail)
 - 픽셀에 의해 생성된 사변형의 가장 긴 변의 길이 사용
 - 픽셀 xy 에 대한 텍셀 uv 의 변화량중 절대값이 가장 큰값



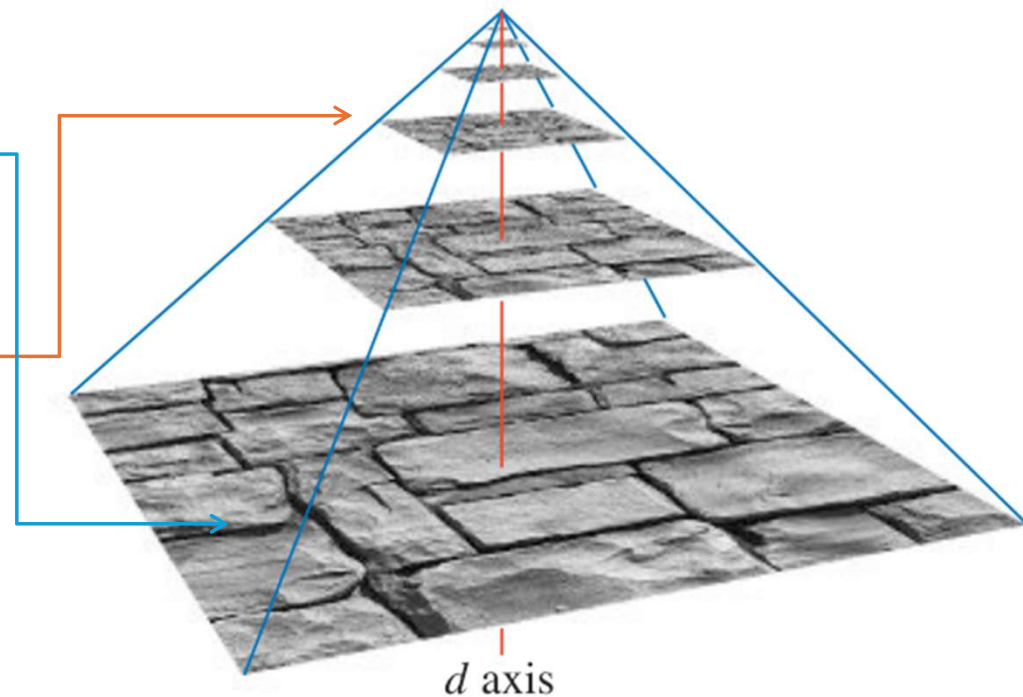
6.2 이미지 텍스처 처리 - 축소

- d 값에 따른 샘플링
- $d = \log_2(\max(\text{abs}(\text{변화량})))$ 레벨
 - ex) 픽셀 하나가 4x4 텍셀을 덮는다면 $d = 2 \rightarrow$ 레벨2 (원본 $\frac{1}{4}$ 크기)
- d가 정수가 아닐 경우 $\rightarrow$ 삼중 선형 보간(trilinear interpolation)
 - 상위 텍스처와 하위 텍스처를 각각 이중 선형 보간하여 컬러 추출
 - 두 개의 색상 세트를 d값에 맞춰 선형 보간

6.2 이미지 텍스처 처리 - 축소

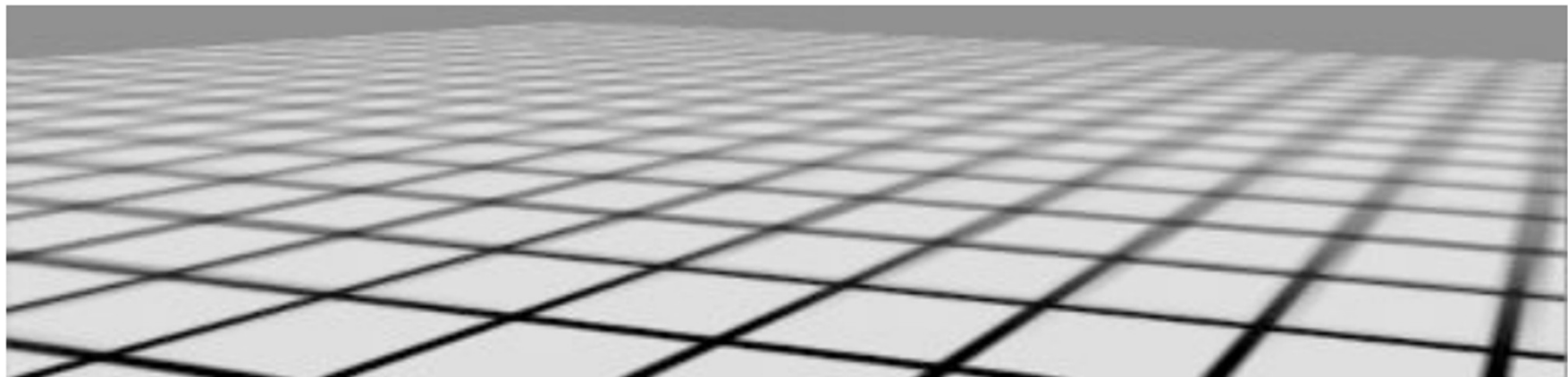
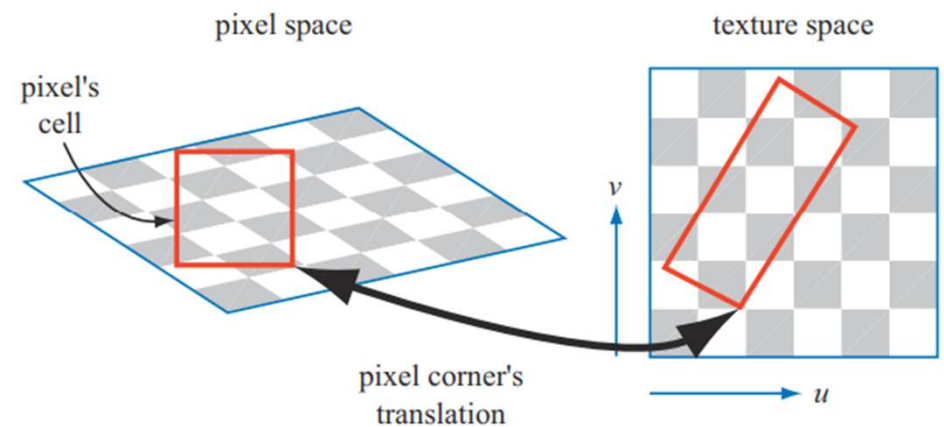
- 상세 수준 편향 값(LOD bias)

- 사용자 제어 가능
- Negative bias : d값을 작게
 - 날카롭고 선명해짐
 - 앨리어싱 발생 가능
- Positive bias : d값을 크게
 - 앨리어싱 사라짐
 - 블러링



6.2 이미지 텍스처 처리 - 축소

- d = 가장 큰 변화량 기준
- 필요 없는 블러링 발생
- 오버블러링(overblurring)



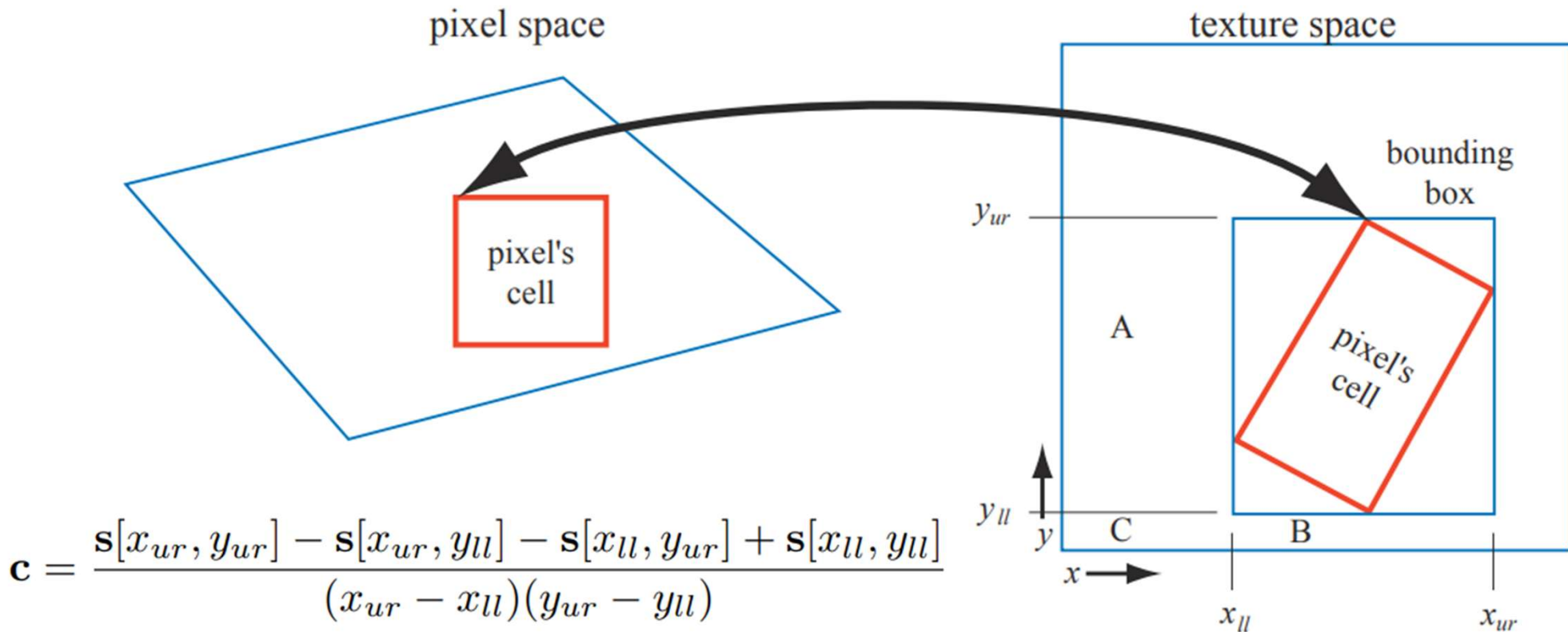
6.2 이미지 텍스처 처리 - 축소

- 합산 영역 테이블(SAT, Summed-Area Table)
 - 2차원 누적합
 - 텍스처와 동일한 크기 배열 생성
 - 숫자가 커지기 때문에 일반 텍스처(8비트) 보다 큰 비트 필요

1	1	1	→	4	6	10
2	1	0		3	3	7
1	0	3		1	1	4

6.2 이미지 텍스처 처리 - 축소

- 합산 영역 테이블(SAT, Summed-Area Table)

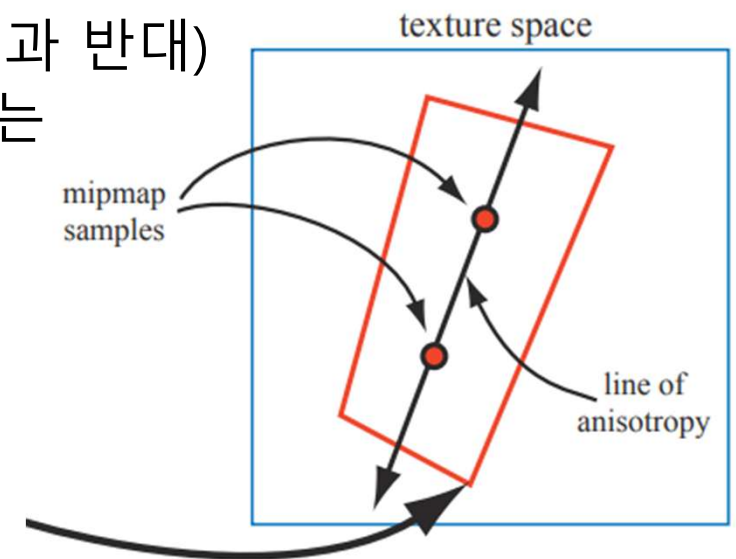


6.2 이미지 텍스처 처리 - 축소

- 합산 영역 테이블(SAT, Summed-Area Table)
- 장점
 - 이방성(비등방) 필터링(Anisotropic filtering)
- 단점
 - 대각선 문제 : 실제 픽셀과 상관 없는 텍셀 포함 -> 오버블러링
 - 메모리 비용 : 높은 정밀도에 따른 많은 비트 수 필요
- 실제 활용 : 넓은 영역 평균값 빠르게 구하기
 - 피사계 심도(depth of field)
 - 부드러운 그림자(shadow maps)
 - 흐릿한 반사(blurry Reflection)

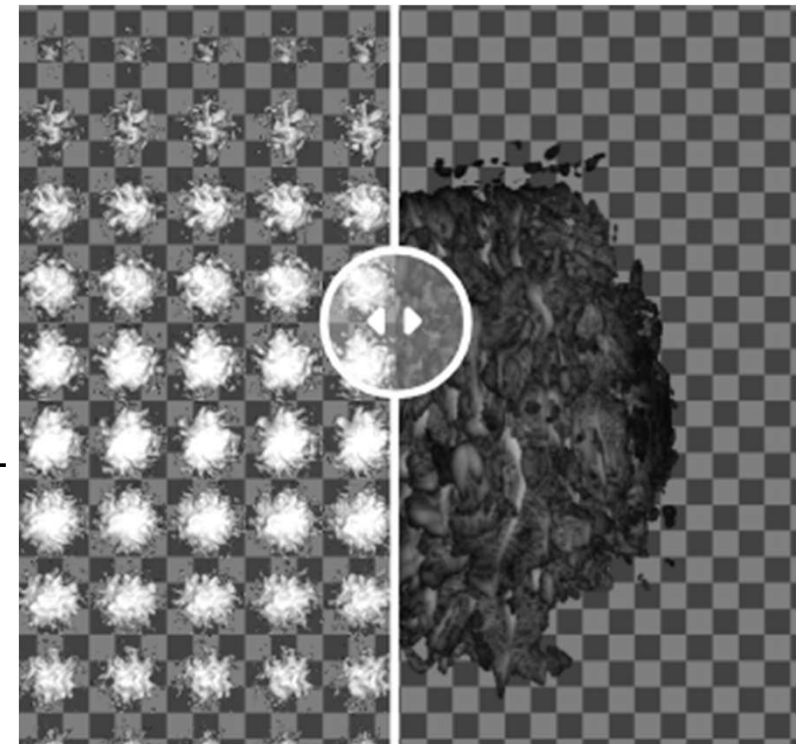
6.2 이미지 텍스처 처리 - 축소

- 제약 없는 이방성(비등방) 필터링
(Unconstrained Anisotropic Filtering)
 - mip맵과 SAT의 단점 모두 극복
 - 정사각형 여러 개로 직사각형 채우기
- 방식
 - 사각형의 짧은 쪽을 사용해 d 결정(기존 mip맵과 반대)
 - 사각형 긴쪽 방향으로 사각형의 중심을 지나는 비등방선(line of anisotropy) 생성
 - 비등방한 정도에 따라 샘플을 생성
 - 샘플마다 기존 mip맵 사용
 - 이방성필터링 : x2 x4 .. x16



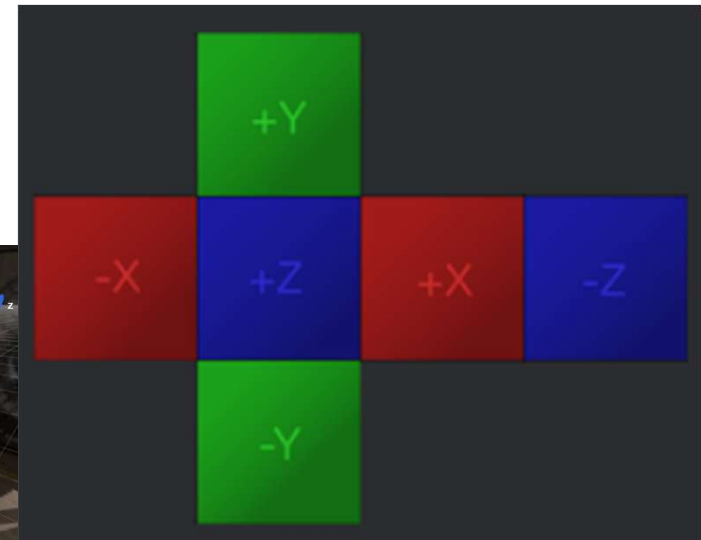
6.2 이미지 텍스처 처리 - 볼륨 텍스처

- 볼륨 텍스처(Volume Textures)
 - 이미지 텍스처의 3차원 확장
 - 깊이 값인 w 를 추가해 (u, v, w) 좌표
- 장점
 - 2차원의 복잡한 매개변수화 과정 생략
 - 매개변수화의 왜곡, 봉합 등 문제 발생 X
 - 목재, 대리석 같은 재질의 볼륨 구조 표현 가능
- 단점
 - 높은 메모리 점유율
 - 비효율적 연산
- 개선
 - 희소 옥트리(sparse octree) 구조 저장



6.2 이미지 텍스처 처리 – 큐브 맵

- 큐브 맵(Cube map / Cube texture)
 - 6개의 정사각형 텍스처로 이루어짐
 - 큐브 중심에서 밖으로 뻗어 나가는 3차원 방향 벡터(x, y, z)로 접근
- 작동 방식
 - 면 선택 : 벡터 성분 중 절대값이 가장 큰 벡터 선택
 - 정규화 : 나머지 두 성분을 위 성분의 절댓값으로 나누기
 - 리매핑 : 이 값 $[-1, 1]$ 을 $[0, 1]$ 로 변환하여 텍스처의 uv 좌표로 사용
- 활용
 - 환경 매핑(environment mapping)
 - 스카이박스(skybox)



6.2 이미지 텍스처 처리 - 텍스처 표현

- 텍스처 표현 기법(Texture Representation)
- 렌더링중 텍스처 교체 -> CPU와 GPU 사이 오버헤드 발생
- 방지를 위한 세가지 주요 기술
 - 텍스처 아틀라스(Texture Atlas)
 - 텍스처 배열(Texture Array)
 - 바인드리스 텍스처(Bindless Textures)

6.2 이미지 텍스처 처리 - 텍스처 표현

- 텍스처 아틀라스(Texture Atlas)
 - 작은 이미지 여러 개를 하나의 큰 텍스처 하나에 모아 넣음
- 장점
 - 여러 오브젝트를 그릴 때 한번만 바인딩 하면 됨
- 단점
 - 타일링 제한 (Repeat 등)
 - mip맵 블리딩(Bleeding) : 하위 레벨로 갈수록 인접한 텍스처의 엉뚱한 색상이 들어올 수 있음
 - 텍스처 사이 padding으로 해결 가능



6.2 이미지 텍스처 처리 - 텍스처 표현

- 텍스처 배열(Texture Array)
 - Atlas의 단점을 해결하기 위해 등장한 API
 - 여러장의 텍스처를 쌓아올린 형태
 - 모든 텍스처의 크기, 포맷 등이 동일해야함
- 장점
 - 개별 바인딩보다 매우 빠름
 - 경계 문제(Repeat 등) 해결



6.2 이미지 텍스처 처리 - 텍스처 표현

- 바인드리스 텍스처(Bindless Textures)
 - 기존에 사용하는 텍스처 유닛(slot)이 아닌 메모리상 주소 값인 64비트 포인터를 셰이더에 직접 전달
- 장점
 - 개수 제한 없음
 - 오버헤드 없음



6.2 이미지 텍스처 처리 - 텍스처 압축

- 텍스처 압축(Texture Compression)
 - 메모리와 대역폭 및 캐시 효율을 직접적으로 올릴 수 있는 방법
 - JPEG, PNG .. 이미지 압축 -> 하드웨어 디코딩 구현 비용 높음
 - 고정 비율 + 랜덤 액세스 필요 (이미지의 특정 부분을 즉시 찾아내 다른 부분과 독립적으로 디코딩 가능해야함)

6.2 이미지 텍스처 처리 - 텍스처 압축

- S3의 S3TC 체계(DXTC / BC)

- 7가지 변형 존재
- DXT는 DirectX 9
- BC는 DirectX 10 이상
- PC 그래픽 표준

- 기본 원리

- 4x4 텍셀 블록 단위로 인코딩
- 블록마다 2개의 대표 색상을 저장
- 16개 텍셀에 두 색상 사이의 보간 계수 기록

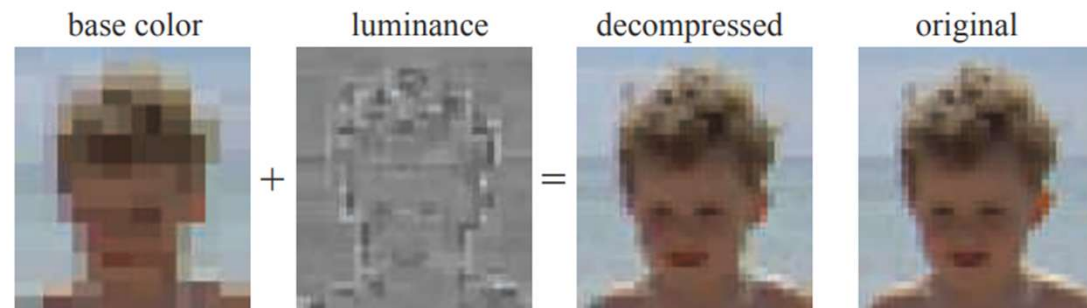
이름	스토리지	참조 컬러	인덱스	알파	비고
BC1/DXT1	8B/4bpt	RGB5652	2bpt	—	1라인
BC2/DXT3	16B/8bpt	RGB5652	2bpt	4bpt raw	BC1과 동일 컬러
BC3/DXT5	16B/8bpt	RGB5652	2bpt	3bpt interp.	BC1과 동일 컬러
BC4	8B/4bpt	R82	3bpt	—	1채널
BC5	16B/8bpt	RG882	2 × 3bpt	—	2BC4
BC6H	16B/8bpt	text 참고	text 참고	—	HDR용; 1~2라인
BC7	8B/4bpt	text 참고	text 참고	optional	1~3라인

6.2 이미지 텍스처 처리 - 텍스처 압축

- BC1/DXT1 – 불투명 이미지
 - 16비트 참조 RGB값 2개
 - 2비트 보간계수
- BC2/DXT3 & BC3/DXT5 – 투명 이미지
 - BC2/DXT3 : BC1 + 알파 값 4비트 저장
 - BC3/DXT5 : BC1 + 알파 값을 색상처럼 두 개 값으로 보간
- BC4&BC5 – 수치 데이터
 - BC4 : 단일 채널(grayscale)
 - BC5 : 노멀 맵 채널 분리
- BC6H
 - HDR, 부동 소수점 데이터
- BC7
 - LDR, 블록마다 여러개 보간 라인

6.2 이미지 텍스처 처리 - 텍스처 압축

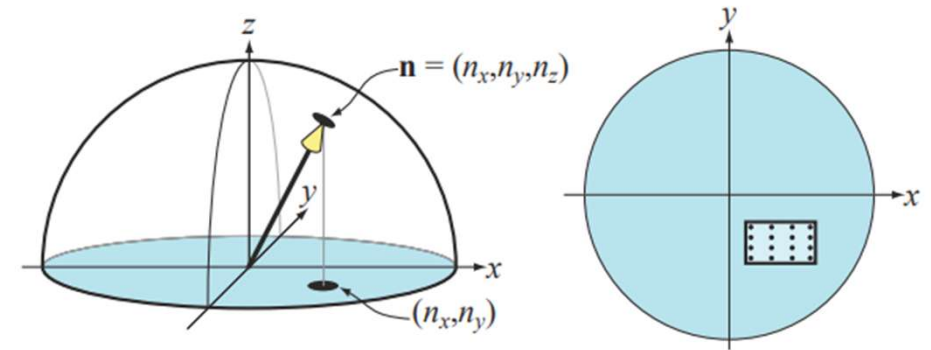
- ETC(Ericsson Texture Compression)
 - 모바일 표준
- ETC1
 - 두개의 4x4 블록을 서브블록으로 분리
 - 각 서브 블록은 하나의 기준 색상을 가짐
 - 하드웨어에 내장된 숫자 테이블에서 값을 가져와 휘도 변경
- ETC2
 - 사용하지 않는 비트 조합 활용 개선
- EAC
 - 알파값 채널



6.2 이미지 텍스처 처리 - 텍스처 압축

- 노말 맵 압축

- $xyz \rightarrow xy + n_z = \sqrt{1 - n_x^2 - n_y^2}$.
- BC5/3Dc에 x와y 독립 채널로 압축
 - 4x4 블록 단위로 수행
 - 각 채널 3비트 보간
- 2개의 EAC 채널
- DXT5(fallback)
 - 정밀도 높은 green과 alpha채널에 저장
- ASTC
 - 4x4에서 12x12까지 가능
 - 1-4개 채널 가능
 - LDR, HDR 텍스처 가능
 - OpenGL ES 3.2 이상 버전에 포함



6.2 이미지 텍스처 처리 - 텍스처 압축

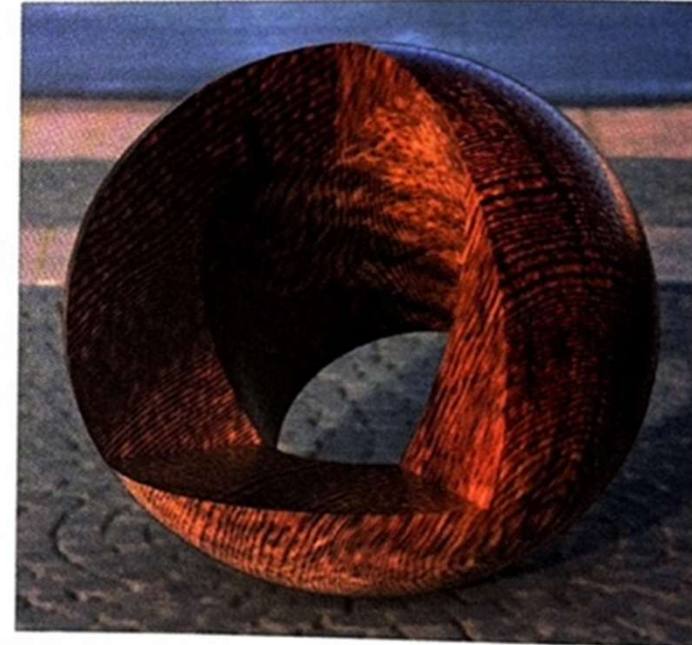
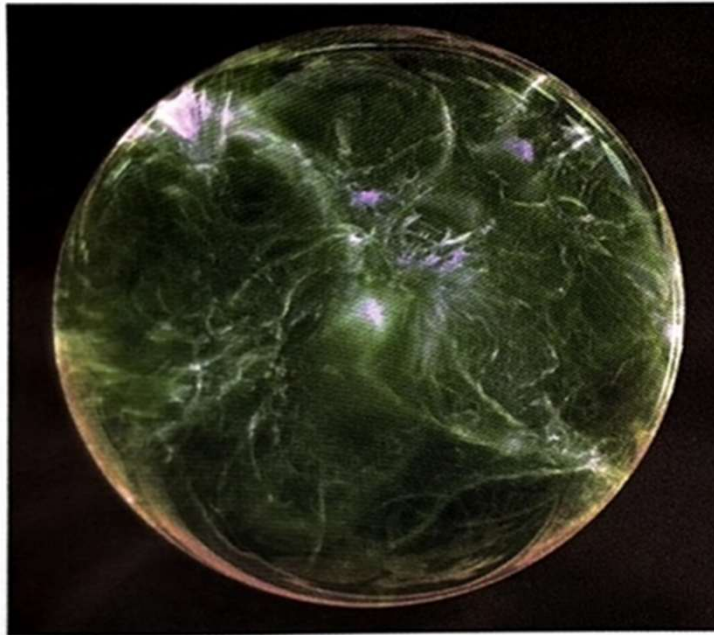
- 압축 텍스처 품질 향상 기법
 - 16비트 성분 사용
 - 히스토그램 재정규화(Histogram renormalization)
 - 압축 전 데이터 분산
 - 대비 향상
 - 밴딩 아티팩트 감소
 - 에러 메트릭(error metric)

$$e = \arccos \left(\frac{\mathbf{n} \cdot \mathbf{n}_c}{\|\mathbf{n}\| \|\mathbf{n}_c\|} \right)$$

- 압축된 노멀($\mathbf{n}_c$)과 원래 노멀($\mathbf{n}$)의 각도차이를 측정

6.3 절차적 텍스처 처리

- 절차적 텍스처 처리(Procedural Texturing)
 - 이미지 파일이 아닌 수학적 연산을 통해 실시간으로 무늬를 만들어 냄
- 장점
 - 용량 절약
 - 볼륨 텍스처
 - 무한한 해상도
 - 동적 반응
 - 물결의 파장 등



6.3 절차적 텍스처 처리

- 노이즈 함수 (Noise function)
 - 옥타브(Octaves) : 노이즈를 여러 주파수로 겹쳐 쌓은 것
낮은 주파수와 높은 주파수가 합쳐져 자연스러움 형성
 - 난류 함수 (Turbulence function) : 가중치가 부여된 노이즈 샘플 들의 합
- 셀룰러 텍스처(Cellular texture)
 - 공간에 흩어져있는 특성점(feature point)들의 사이의 거리를 측정해, 색이나 노멀 값으로 변환하여 무늬를 만듦
 - 파충류 피부, 세포 등
- 절차적 텍스처 안티앨리어싱
 - mip맵을 만들 수 없음
 - 대역 제한(band limiting) : 프로그래머가 노이즈의 주파수를 알고 있기 때문에 주파수 대역을 제한하여 방지

6.4 텍스처 애니메이션

- 동적 소스 활용
 - 비디오 소스
- 텍스처 좌표 애니메이션(uv animation)
 - 텍스처 좌표를 변화시킴
 - uv 스크롤링(uv scrolling) : 폭포수처럼 맵프레임 일정 값 증감
 - 변환 행렬 : 회전, 확대, 전단 등
 - 워핑&모핑(warping&morphing) : 수학적 함수를 이용해 uv 좌표를 비틀
- 텍스처 블렌딩(Texture Blending)
 - 두 개 이상의 텍스처를 블렌딩하여 전환

6.5 재질 매핑

- 재질 매핑(Material mapping)

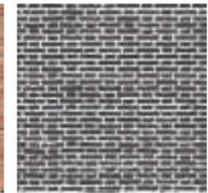
- 픽셀 셰이더가 텍스처에서 값을 읽어와 매개변수를 실시간으로 수정
- 알베도(Albedo/Diffuse) : 표면의 색상을 지정
- 거칠기, 고도, 메탈릭, 투명도 등

- 필터링과 엘리어싱

- 선형 관계 : mipmap, 선형 필터링 가능
 - 표면 컬러
- 비선형 관계 : 일반적인 방법으로 불가능 -> PBR
 - 거칠기, 범프 매핑



albedo
texture



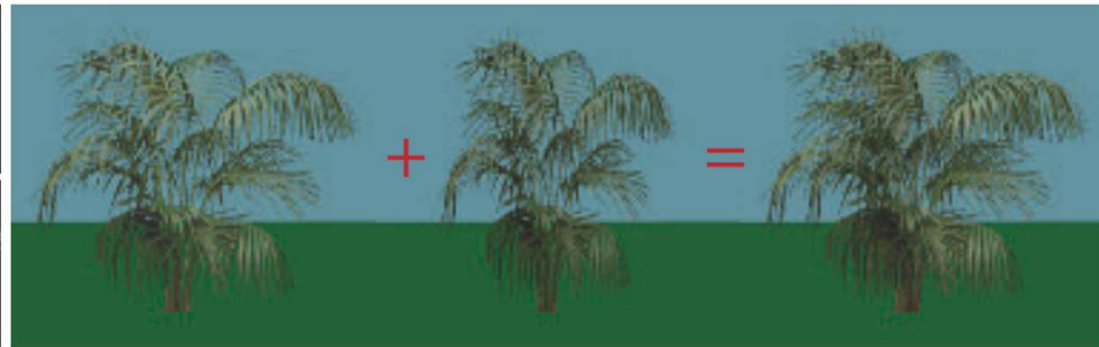
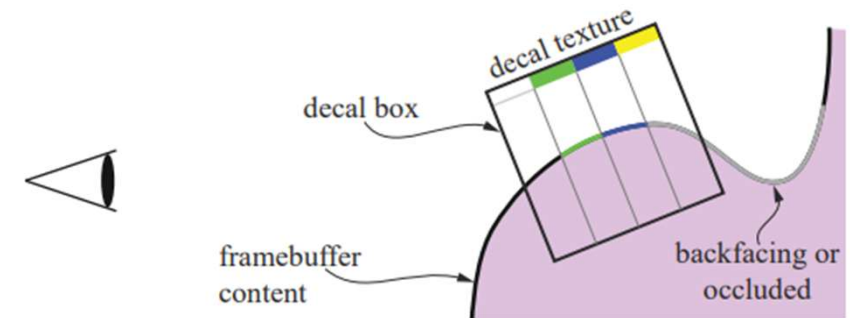
roughness
texture



heightfield
texture

6.6 알파 매핑

- 알파 매핑(Alpha mapping)
 - 데칼(decals) -> ch.20
 - 스티커 붙이기
 - 컷아웃(cutout)
 - 단순한 폴리곤에 텍스처입히기
 - 회전에서 문제
 - 빌보딩(billboarding) -> ch.13
 - 2D이미지가 항상 카메라 바라보게 회전시키기



6.6 알파 매핑 – 렌더링 방식

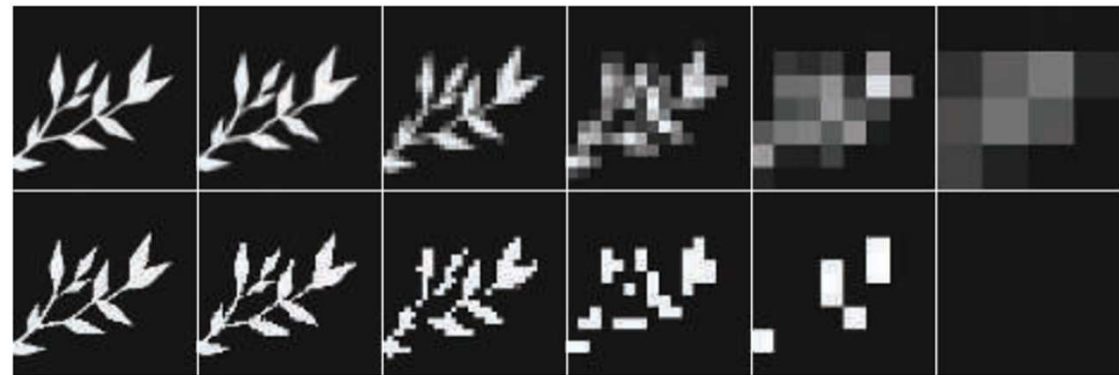
- 알파 블렌딩(alpha blending)

- 반투명 가능
- back-to-front 렌더링
- 비효율적

- 알파 테스트(alpha testing)

```
if (texture.a < alphaThreshold) discard;
```

- 0-1 렌더링
- 임계값 설정
- 정렬 순서 상관 없이 렌더링
- 확대/축소 문제



각 렌더링 방식의 맵

6.6 알파 매핑 - 렌더링 방식

- 알파 테스트 + mip맵
 - 알파값 평균화

- 커버리지 보존

$$c_k = \frac{1}{n_k} \sum_i (\alpha(k, i) > \alpha_t)$$

- n_k : mip맵 레벨 k에서 텍셀수
- $a(k, i)$: mip맵 레벨 k에서 픽셀 i의 알파 값
- a_t : 알파 임계값
- c_k : mip맵 레벨 k에서 테스트 통과 비율



6.6 알파 매핑 – 렌더링 방식

- 확률적 보존

```
if (texture.a < random()) discard;
```

- 임계값을 고정하지 않고 확률로 처리
- ex) 알파 값 0.3 -> 30% 확률 그려짐
- 실구현은 노이즈를 방지하기 위해 해시 함수 사용

- Alpha to Coverage

- MSAA 활용
- 알파 값에 따라 일부 샘플만 활성화

fwidth()



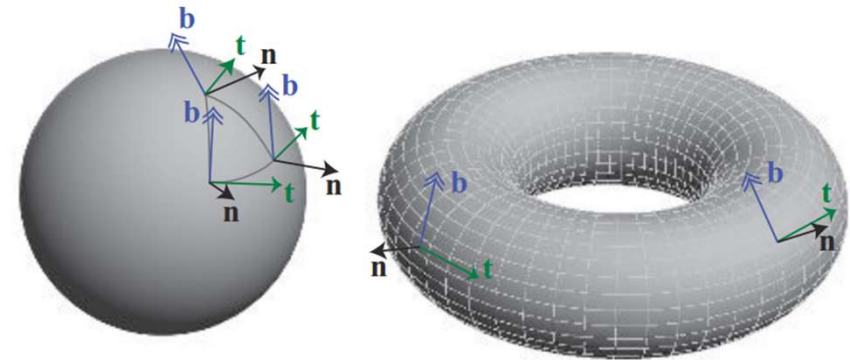
6.7 뱀프 매핑

- 뱀프 매핑(Bump mapping)
 - 정점을 추가하지 않고 텍스처 매핑만으로 보다 나은 결과를 제공하는 기법
 - 픽셀별 음영처리 루틴을 수정
- 주요 기법
 - 블린의 방법(Blinn's Methods)
 - 노멀 매핑(Normal mapping)

6.7 범프 매핑

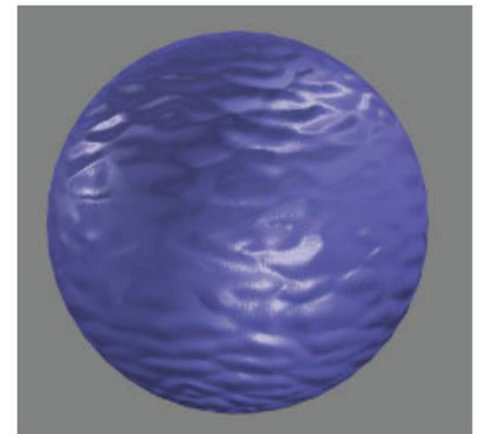
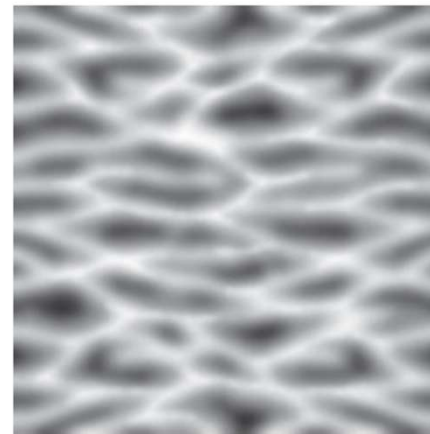
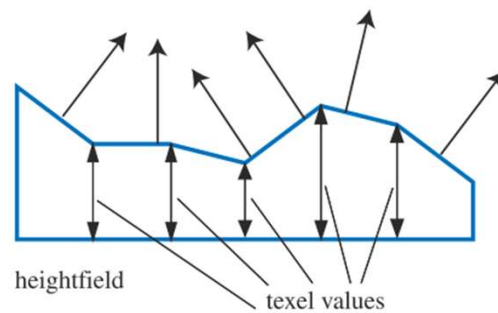
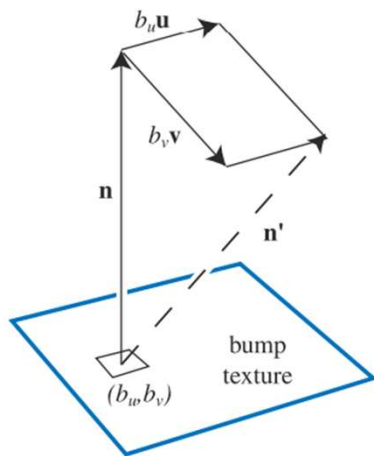
- 탄젠트 공간(tangent space)
 - 표면의 굴곡 정보를 저장하기 위한 표면 기준 좌표계
 - T(Tangent, 접선) : 표면의 가로(u) 방향과 일치
 - B(Bitangent, 종법선?) : 표면의 세로(v) 방향과 일치
 - N(Normal, 법선) : 표면에서 뺀어 나오는 축
- TBN 행렬
 - 탄젠트 공간과 월드 공간의 변환을 도와주는 기저 행렬

$$\begin{pmatrix} t_x & t_y & t_z & 0 \\ b_x & b_y & b_z & 0 \\ n_x & n_y & n_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



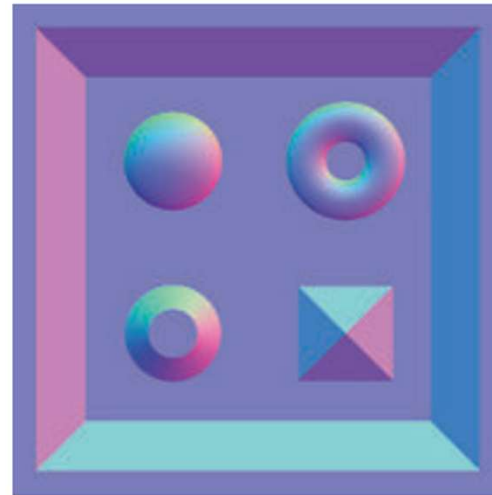
6.7 범프 매핑

- 블린의 방법(Blinn's methods)
 - 오프셋 맵(offset map) : 텍셀마다 2개의 부호 있는 값 b_u , b_v 를 이용해 원래 법선의 방향을 바꿈
 - 높이 필드(Heightfield) : 단색 텍스처를 이용하여 인접 텍셀 간의 차이(경사)를 통해 법선을 유도



6.7 범프 매핑

- 노멀 매핑(Normal mapping)
 - 법선 벡터를 텍스처의 RGB 채널에 직접 저장하는 방식
 - 블린의 방법과 수학적으로 거의 동일한 결과
 - 0~255 값을 -1.0~1.0 범위로 매핑
 - 텍스처 재사용과 압축 효율 좋음



6.7 범프 매핑

- 노멀 맵 필터링

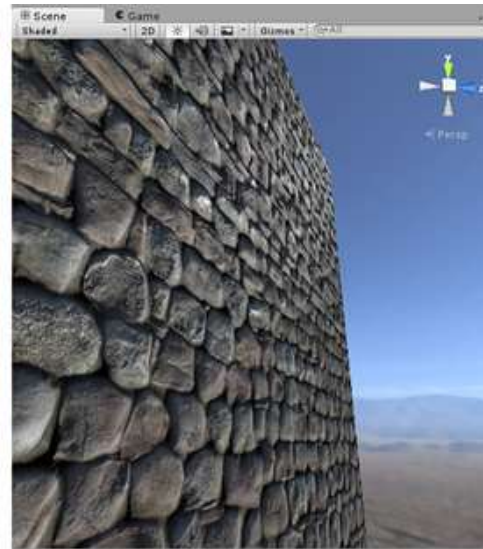
- 일반 텍스처와 달리 필터링 시 비선형적 문제 발생
 - 하이라이트 깨짐, sparkle 현상, 어두워짐
- 램버시안(Lambertian) 표면 : 내적이 주를 이루므로 선형적 연산 가능(ex: 달)

$$\mathbf{l} \cdot \left(\frac{\sum_{j=1}^n \mathbf{n}_j}{n} \right) = \frac{\sum_{j=1}^n (\mathbf{l} \cdot \mathbf{n}_j)}{n}.$$

- 음수 값을 막기 위한 $\max(\mathbf{l} \cdot \mathbf{n}, 0)$ 연산을 수행하여 비선형성 존재
- 비-램버시안 표면 -> ch.9

6.8 시차 매핑

- 시차 매핑(Parallax mapping)
 - 평면에 실제 높이가 있는 것처럼 속이는 기법
 - 시선 벡터가 높이 필드(Heightfield)와 부딪히는 지점을 찾아내어 텍스처 좌표를 이동 시킴

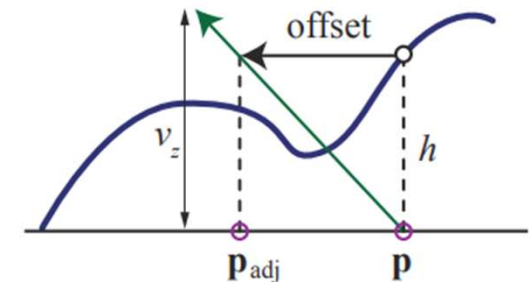
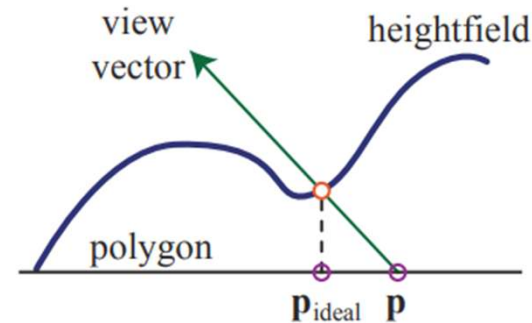


6.8 시차 매핑

• 작동 원리

$$\mathbf{p}_{\text{adj}} = \mathbf{p} + \frac{h \cdot \mathbf{v}_{xy}}{v_z}$$

- $\mathbf{p}$: 원래 텍스처 좌표
- h : 해당 지점 높이 필드 값
- $\mathbf{v}_{xy}$: 시선 벡터의 수평 성분
- v_z : 높이 값



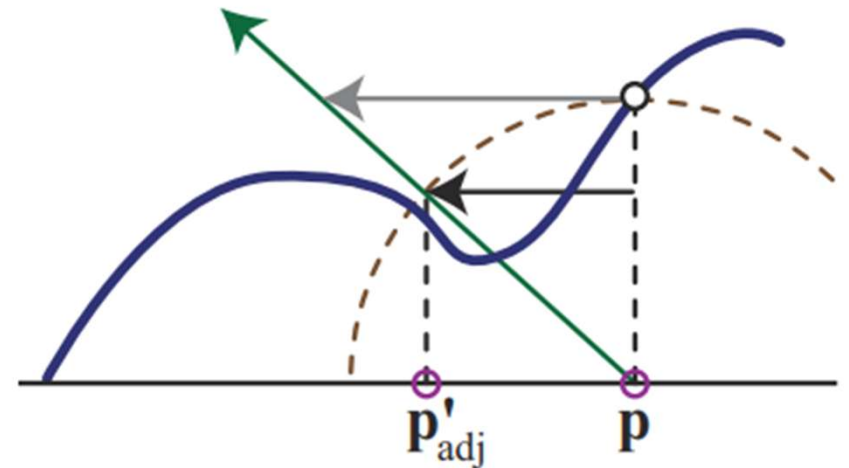
- $\mathbf{p}$ 대신 $\mathbf{p}_{\text{ideal}}$ 근처의 색상을 끌어오기
- 높이(h)가 높을수록 클수록, 시선이(v_z)표면과 평행할수록 더 많이
- 얇은 시야각에서 문제 발생

6.8 시차 매핑

- 오프셋 제한

$$\mathbf{p}'_{\text{adj}} = \mathbf{p} + h \cdot \mathbf{v}_{xy}$$

- 오프셋이 h 보다 커지지 않도록 제한



6.8 시차 매핑

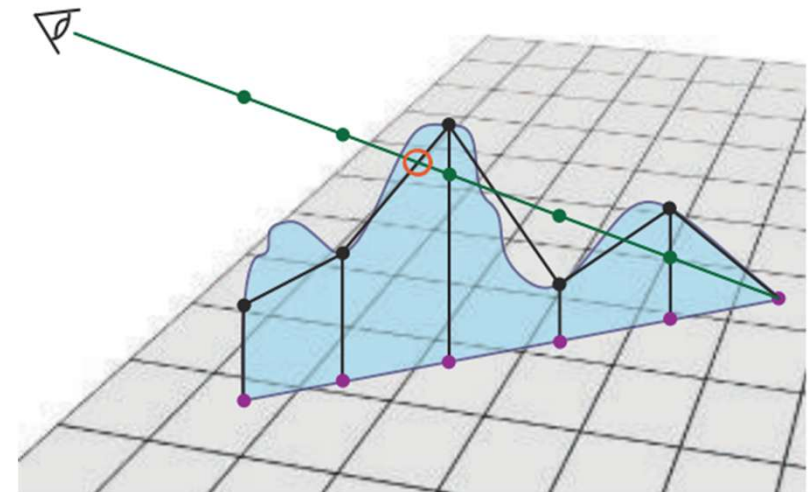
- 시차 폐색 매핑(parallax occlusion mapping, POM)
 - 기본 시차 매핑은 인접 픽셀의 높이가 비슷하다는 가정
 - 급격한 경사 부분에서 self-occlusion과 그림자 불가능
 - 시선 벡터를 따라 높이 필드를 가로지르며 실제 교차점 찾기



6.8 시차 매핑

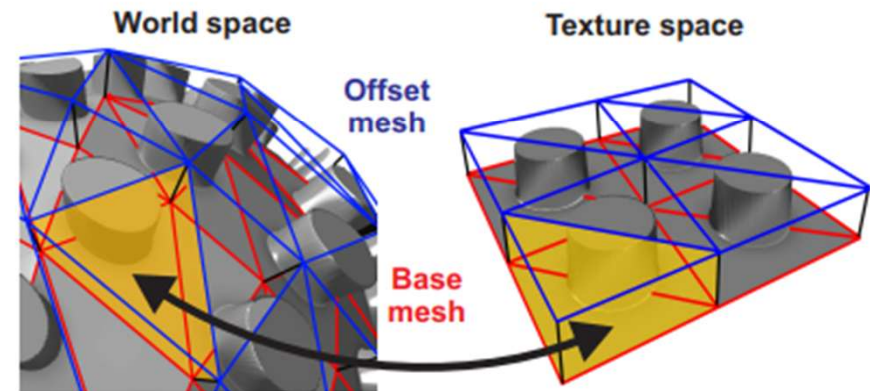
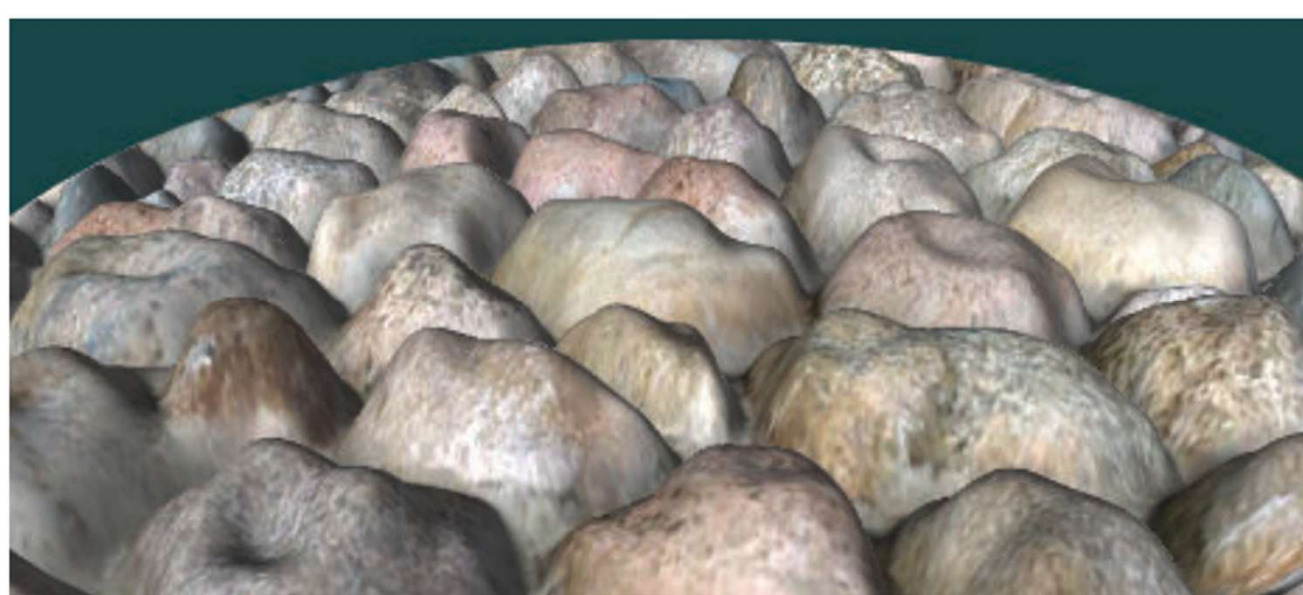
- Ray marching

- 카메라에서 픽셀을 향해 나가는 뷰 벡터 정의
- 뷰 벡터를 여러 단계로 나누어 전진
- 샘플링된 heightfield의 값과 내 뷰 벡터의 높이를 비교
- heightfield가 더 높을 경우 교차(Intersection) 발생 간주
 - 정교한 위치를 찾기위해 이분탐색,할선법(secant method) 등 사용
- 교차점의 uv좌표로 현재 텍스처 좌표 대체



6.8 시차 매핑

- POM 한계
 - 실루엣 문제
 - 2차 실루엣 근사 기법 : 가장자리 곡률을 수학적으로 근사
 - shell mapping : 기본 메쉬 위에 껍데기 영역을 생성하여 입체감 구성



6.9 텍스처 조명

- 텍스처를 광원에 사용하여 복잡한 빛의 강도 분포나 스포트라이트 등 구현 가능

